

HTTP Proxy - Explicații și Pași către Produsul Final

1. Scopul proiectului

Aplicația reprezintă un sistem format din trei componente principale:

- client: trimite cereri HTTP către proxy;
- proxy: interceptează cererile, le poate modifica, bloca sau analiza;
- server: răspunde cu un mesaj HTTP.

Fluxul de date este: client → proxy → server → proxy → client.

Scopul final: un proxy HTTP interactiv care interceptează traficul, îl afișează și permite operatorului să modifice cererile și răspunsurile.

2. Codul de bază (scheletul actual)

1. server.c – trimite un mesaj HTML simplu pe portul 8000.
2. proxy.c – primește cereri de la client, le redirecționează către server și returnează răspunsul.
3. client.c – trimite o cerere completă către proxy.

3. Rolul fiecărei componente

Componentă	Rol	Port	Conectat la
-----	----	----	-----
client.c	Trimite cereri către proxy	—	proxy (8080)
proxy.c	Interceptează, modifică, redirecționează	8080	server (8000)

4. Cum se rulează

1. Compilează toate fişierele:

make

2. Rulează în 3 terminale:

./server

./proxy

./client

3. Verifică:

- clientul afişează textul HTML;
- proxy-ul afişează cererea şi mesajul „Forwarded response”;
- serverul arată cererea completă primită.

5. Ce mai trebuie implementat

a) Parsare HTTP completă

- extragerea method, URL, host, body etc.
- suport GET/POST.

b) Engine de reguli (rules.txt)

- reguli aplicabile pe request/response:

block_url request .*forbidden.*

header_replace request User-Agent .* MyProxy/1.0

body_replace response Hello Salut

c) Mod interactiv

- operatorul alege acțiunea:

f – forward, b – block, e – edit header, r – replace în body, s – salvează regula.

d) Logging

- log în logs/proxy.log:

[2025-10-30 14:21:00] [conn 2] GET http://localhost:8000 -> 200 OK [applied: none]

e) Suport CONNECT (HTTPS)

- detectează metoda CONNECT;
- creează tunel TCP între client și server;
- nu decriptează conținutul TLS.

f) Funcționalități opționale

- reload reguli (SIGHUP);
- admin socket;
- mascarea datelor sensibile (Cookie, Authorization).

6. Testare

1. Local: toate procesele pe aceeași mașină.

2. În rețea: proxy-ul ascultă pe IP rețea (192.168.x.x), clientul trimite cereri acolo.

3. Reguli:

- block: client primește 403;
- replace: textul e modificat;
- header_replace: serverul primește User-Agent nou.

7. Rezultat final

Produsul complet trebuie să:

- funcționeze cu clientul și serverul proprii;
- poată intercepta, bloca și modifica traficul HTTP;
- folosească fișier de reguli;
- aibă mod interactiv și loguri.